

EN3160 Assignment 1 — Intensity Transformations and Neighborhood Filtering

Name: Perera W.A.L.S.

Index Number: 230487D

GitHub: <https://github.com/Lasan-Perera/EN3160-assignment-1>

Question 1: Piecewise Linear Intensity Transformation

Implement `intensity_transform(im, breakpoints)` and apply it to the given image with the breakpoints from the assignment. Experiment with different breakpoints for a visually pleasing result.

```
In [11]: import cv2
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 80
plt.rcParams['savefig.dpi'] = 80
plt.rcParams['savefig.bbox'] = 'tight'

def intensity_transform_step(im):
    lut = np.zeros(256, dtype=np.uint8)

    # 0 <= x < 50 : y = x
    lut[0:50] = np.arange(0, 50)

    # 50 <= x < 150 : y maps from 100 to 255
    lut[50:150] = np.linspace(100, 255, 100, endpoint=False).astype(np.uint8)

    # 150 <= x <= 255 : y = x
    lut[150:256] = np.arange(150, 256)

    transformed_im = cv2.LUT(im, lut)
    return transformed_im, lut

# read the image in grayscale
img_gray = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/emma.jpg',
cv2.IMREAD_GRAYSCALE)

# apply the transformation
transformed_img_gray, lut = intensity_transform_step(img_gray)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

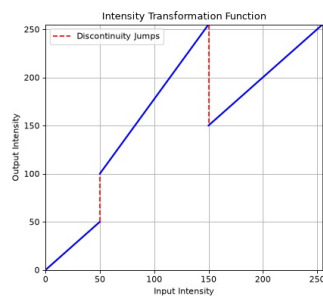
# plot piecewise step transformation
axes[0].plot([0, 50], [0, 50], 'b-', linewidth=2)
axes[0].plot([50, 50], [50, 100], 'r--', linewidth=1.5, label='Discontinuity Jumps')
axes[0].plot([50, 150], [100, 255], 'b-', linewidth=2)
axes[0].plot([150, 150], [255, 150], 'r--', linewidth=1.5)
axes[0].plot([150, 255], [150, 255], 'b-', linewidth=2)

axes[0].set_title('Intensity Transformation Function')
axes[0].set_xlabel('Input Intensity')
axes[0].set_ylabel('Output Intensity')
axes[0].set_xlim([0, 255])
axes[0].set_ylim([0, 255])
axes[0].grid(True)
axes[0].legend()

# original Image
axes[1].imshow(img_gray, cmap='gray', vmin=0, vmax=255)
axes[1].set_title('Original Grayscale Image')
axes[1].axis('off')

# transformed Image
axes[2].imshow(transformed_img_gray, cmap='gray', vmin=0, vmax=255)
axes[2].set_title('Transformed Image')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```



Discussion

The midcolourtones are stretched so the face gains contrast but shows hard bright bands where the original has no edge. In this we can see input 149 maps brighter than 150 because of that the band sits directly against a darker region instead of blending into it.

Question 2: Accentuating White and Gray Matter

Apply piecewise linear intensity transformations to the brain proton density image to accentuate (a) white matter and (b) gray matter separately.

Brain Tissue Intensity Thresholds

For this implementation, we use the following 8-bit intensity thresholds:

- **0-70:** CSF
- **70-140:** Gray Matter (GM)
- **140-255:** White Matter (WM)

These thresholds are used as practical segmentation limits. MRI intensity values can vary between scanners and acquisition protocols, so these values are specific to our image-processing implementation rather than universal MRI thresholds.

Source: [Robitaille et al., 2012 — Tissue-Based MRI Intensity Standardization](#)

```
In [12]: def apply_custom_lut(img, breakpoints):
    breakpoints = np.array(breakpoints)
    x_pts = breakpoints[:, 0]
    y_pts = breakpoints[:, 1]

    input_range = np.arange(256)
    lut = np.interp(input_range, x_pts, y_pts).astype(np.uint8)

    transformed = cv2.LUT(img, lut)
    return transformed, lut

# Read the brain MRI image in grayscale
brain_img = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/brainProton.jpeg',
cv2.IMREAD_GRAYSCALE)

# Define breakpoints for White Matter (WM: 140-255)
breakpoints_wm = [
    [0, 0],
    [180, 0],
    [181, 255],
    [255, 255]
]

# Define breakpoints for Gray Matter (GM: 70-140)
breakpoints_gm = [
    [0, 0],
    [100, 0],
    [101, 255],
    [200, 255],
    [201, 0],
    [255, 0]
]

# Apply transformations
img_wm, lut_wm = apply_custom_lut(brain_img, breakpoints_wm)
img_gm, lut_gm = apply_custom_lut(brain_img, breakpoints_gm)

# Visualisation
```

```

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# White Matter Transformation Plot
axes[0, 0].plot([p[0] for p in breakpoints_wm], [p[1] for p in breakpoints_wm], 'r-o')
axes[0, 0].set_title('White Matter Transformation Plot')
axes[0, 0].set_xlabel('Input Intensity')
axes[0, 0].set_ylabel('Output Intensity')
axes[0, 0].grid(True)

axes[0, 1].imshow(brain_img, cmap='gray')
axes[0, 1].set_title('Original Brain MRI')
axes[0, 1].axis('off')

axes[0, 2].imshow(img_wm, cmap='gray')
axes[0, 2].set_title('Accentuated White Matter')
axes[0, 2].axis('off')

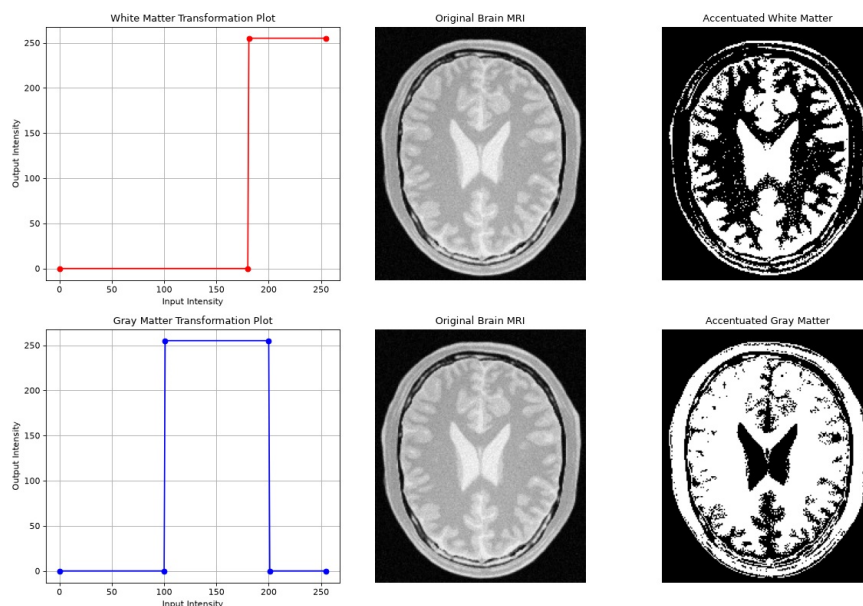
# Gray Matter Transformation Plot
axes[1, 0].plot([p[0] for p in breakpoints_gm], [p[1] for p in breakpoints_gm], 'b-o')
axes[1, 0].set_title('Gray Matter Transformation Plot')
axes[1, 0].set_xlabel('Input Intensity')
axes[1, 0].set_ylabel('Output Intensity')
axes[1, 0].grid(True)

axes[1, 1].imshow(brain_img, cmap='gray')
axes[1, 1].set_title('Original Brain MRI')
axes[1, 1].axis('off')

axes[1, 2].imshow(img_gm, cmap='gray')
axes[1, 2].set_title('Accentuated Gray Matter')
axes[1, 2].axis('off')

plt.tight_layout()
plt.show()

```



Discussion

Each transform's output is binary. The WM result shows the ventricles and the GM result shows the cortical ribbon with the CSF dropping out as a dark centre. Because of this it is easy to identify tumours and other defects.

Question 3: Gamma Correction on the L Plane (Lab*)

Convert to Lab*, apply gamma correction to the L plane, state the gamma value used, and compare histograms before and after correction.

```

In [13]: img_bgr = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/q3GammaCorr.jpeg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Convert to L*a*b* color space
img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
L, a, b = cv2.split(img_lab)

# Apply Gamma Correction to L plane
# gamma value < 1.0 brightens dark areas

```

```
# gamma value > 1.0 darkens bright areas
gamma = 0.2
L_normalized = L / 255.0
L_corrected = np.uint8(np.clip((L_normalized ** gamma) * 255, 0, 255))

# Merge corrected L channel with original a and b channels, then convert back to RGB
img_lab_corrected = cv2.merge([L_corrected, a, b])
img_bgr_corrected = cv2.cvtColor(img_lab_corrected, cv2.COLOR_LAB2BGR)
img_rgb_corrected = cv2.cvtColor(img_bgr_corrected, cv2.COLOR_BGR2RGB)

print(f"Applied Gamma Value is {gamma}")

# Visualisation
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Original Image
axes[0, 0].imshow(img_rgb)
axes[0, 0].set_title("Original Image")
axes[0, 0].axis("off")

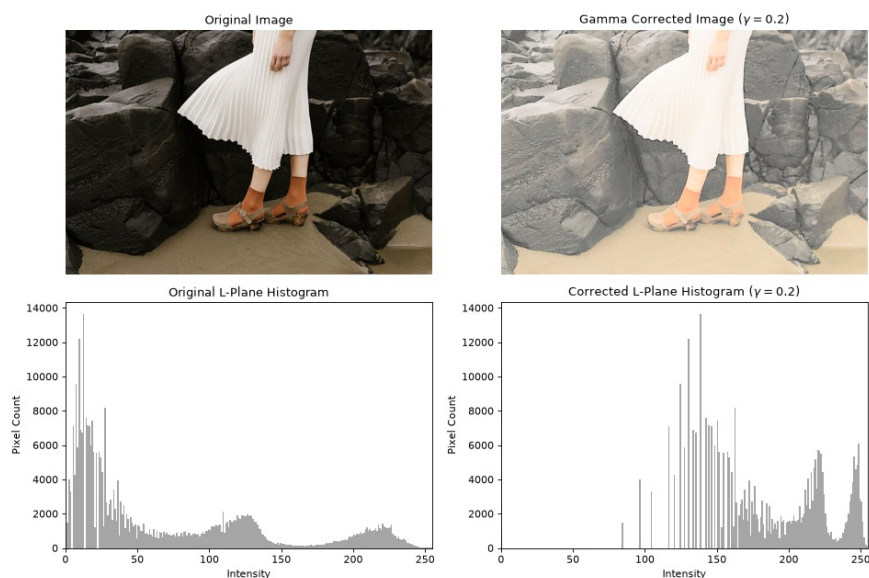
# Gamma Corrected Image
axes[0, 1].imshow(img_rgb_corrected)
axes[0, 1].set_title(rf"Gamma Corrected Image ($\gamma = {gamma}$)")
axes[0, 1].axis("off")

# Original L-channel Histogram
axes[1, 0].hist(L.ravel(), bins=256, range=[0, 256], color='gray', alpha=0.7)
axes[1, 0].set_title("Original L-Plane Histogram")
axes[1, 0].set_xlabel("Intensity")
axes[1, 0].set_ylabel("Pixel Count")
axes[1, 0].set_xlim([0, 255])

# Gamma Corrected L-channel Histogram
axes[1, 1].hist(L_corrected.ravel(), bins=256, range=[0, 256], color='gray', alpha=0.7)
axes[1, 1].set_title(rf"Corrected L-Plane Histogram ($\gamma = {gamma}$)")
axes[1, 1].set_xlabel("Intensity")
axes[1, 1].set_ylabel("Pixel Count")
axes[1, 1].set_xlim([0, 255])

plt.tight_layout()
plt.show()
```

Applied Gamma Value is 0.2

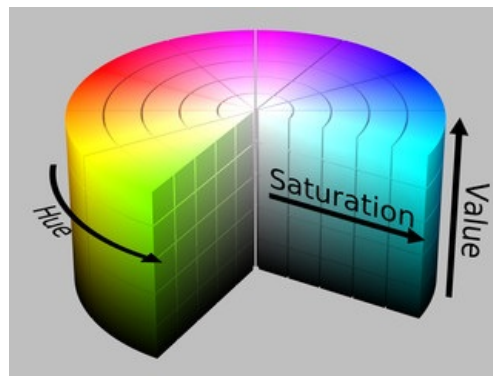


Discussion

$\gamma = 0.2$ lifts up the shadows so hard that the dark peak moves from 5-40 up to 120-165 while the highlights are compressed into 235-250, leaving the bottom third of the histogram empty because of that the image is much brighter but washed out.

Question 4: Vibrance Enhancement via Saturation Plane

Split into HSV planes, apply the given Gaussian-weighted saturation boost $f(x)$ with chosen a and $\sigma=70$, recombine, and display results.



```
In [14]: def vibrance_transform(s_channel, a, sigma=70):
    x = np.arange(256, dtype=np.float32)
    boost = a * 128.0 * np.exp(-((x - 128.0) ** 2) / (2.0 * (sigma ** 2)))
    fx = np.clip(x + boost, 0, 255).astype(np.uint8)
    s_enhanced = cv2.LUT(s_channel, fx)
    return s_enhanced, x, fx

img_bgr = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/SpiderMan.jpeg')
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# Convert to HSV color space and split channels
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(img_hsv)

# parameter for vibrance enhancement
a_param = 0.5
print(f"Reported chosen parameter value: a = {a_param}")
s_enhanced, x_vals, fx_vals = vibrance_transform(s, a=a_param, sigma=70)

img_hsv_enhanced = cv2.merge([h, s_enhanced, v])
img_rgb_enhanced = cv2.cvtColor(img_hsv_enhanced, cv2.COLOR_HSV2RGB)

# Visualisation
fig = plt.figure(figsize=(16, 14))
gs = fig.add_gridspec(3, 4)

# HSV Planes and Transformation Curve
ax_h = fig.add_subplot(gs[0, 0])
ax_h.imshow(h, cmap='gray')
ax_h.set_title("Hue Plane (H)")
ax_h.axis("off")

ax_s = fig.add_subplot(gs[0, 1])
ax_s.imshow(s, cmap='gray')
ax_s.set_title("Original Saturation Plane (S)")
ax_s.axis("off")

ax_v = fig.add_subplot(gs[0, 2])
ax_v.imshow(v, cmap='gray')
ax_v.set_title("Value Plane (V)")
ax_v.axis("off")

# Curve plot spanning 1 column
ax_curve = fig.add_subplot(gs[0, 3])
ax_curve.plot(x_vals, x_vals, 'k--', alpha=0.5, label='y=x')
ax_curve.plot(x_vals, fx_vals, 'r-', linewidth=2, label=rf'$a={a_param}$')
ax_curve.set_title(rf"Transformation Curve ($a={a_param}$)")
ax_curve.set_xlabel("Input S")
ax_curve.set_ylabel("Output f(S)")
ax_curve.set_xlim([0, 255])
ax_curve.set_ylim([0, 255])
ax_curve.grid(True)
ax_curve.legend()

# before & after side-by-side
ax_orig = fig.add_subplot(gs[1, 0:2])
ax_orig.imshow(img_rgb)
ax_orig.set_title("Original Image (RGB)")
ax_orig.axis("off")

ax_enh = fig.add_subplot(gs[1, 2:4])
ax_enh.imshow(img_rgb_enhanced)
ax_enh.set_title(rf"Vibrance Enhanced Image ($a={a_param}$)")
ax_enh.axis("off")

# Effect of Varying Parameter 'a'
```

```

a_values = [0.0, 0.3, 0.6, 1.0]

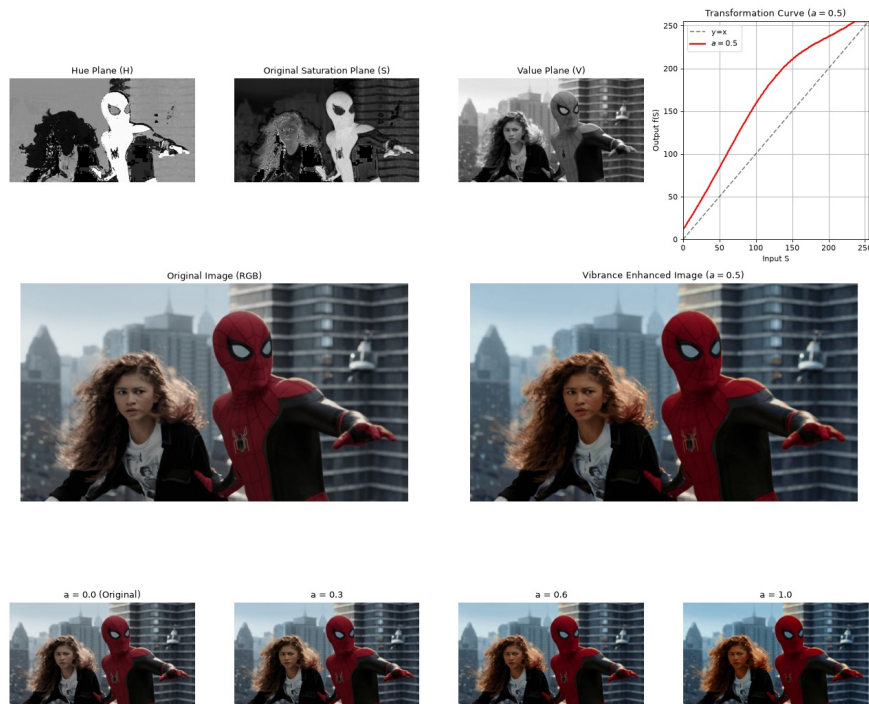
for idx, a_val in enumerate(a_values):
    ax_a = fig.add_subplot(gs[2, idx])
    s_test, _, _ = vibrance_transform(s, a=a_val, sigma=70)
    hsv_test = cv2.merge([h, s_test, v])
    rgb_test = cv2.cvtColor(hsv_test, cv2.COLOR_HSV2RGB)

    ax_a.imshow(rgb_test)
    if a_val == 0.0:
        ax_a.set_title(f"a = {a_val} (Original)")
    else:
        ax_a.set_title(f"a = {a_val}")
    ax_a.axis("off")

plt.tight_layout()
plt.show()

```

Reported chosen parameter value: a = 0.5



Discussion

Mid-saturation regions such as the sky, skin and hair gain up to 64 levels while the already saturated red suit and the neutral grey buildings barely change, so the picture looks richer without the flat over-saturated patches a global boost would give. At $a = 1.0$ the curve stops increasing and tonal separation in the mid saturation range starts to flatten.

Question 5: Histogram Equalization (Custom Implementation)

Write your own histogram equalization function (no `cv.equalizeHist`) and show histograms before/after.

```

In [15]: def custom_histogram_equalization(img):

    if len(img.shape) == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    num_pixels = img.size

    hist, _ = np.histogram(img.flatten(), bins=256, range=[0, 256])

    pdf = hist / num_pixels
    cdf = np.cumsum(pdf)
    transform_map = np.round(255 * cdf).astype(np.uint8)

    equalized_img = cv2.LUT(img, transform_map)
    eq_hist, _ = np.histogram(equalized_img.flatten(), bins=256, range=[0, 256])

    return equalized_img, hist, eq_hist, transform_map

img_gray = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/histogramPic2.jpg',

```

```

cv2.IMREAD_GRAYSCALE)

eq_img, orig_hist, eq_hist, lut_map = custom_histogram_equalization(img_gray)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Original Image
axes[0, 0].imshow(img_gray, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title("Original Image")
axes[0, 0].axis("off")

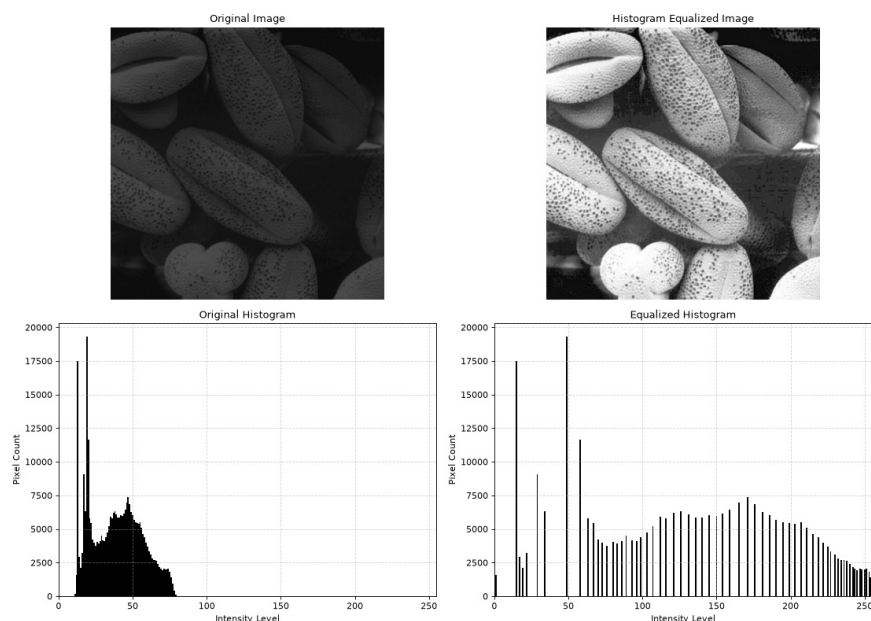
# Equalized Image
axes[0, 1].imshow(eq_img, cmap='gray', vmin=0, vmax=255)
axes[0, 1].set_title("Histogram Equalized Image")
axes[0, 1].axis("off")

# Original Histogram
axes[1, 0].bar(range(256), orig_hist, color='black', width=1.0)
axes[1, 0].set_title("Original Histogram")
axes[1, 0].set_xlabel("Intensity Level")
axes[1, 0].set_ylabel("Pixel Count")
axes[1, 0].set_xlim([0, 255])
axes[1, 0].grid(True, linestyle='--', alpha=0.5)

# Equalized Histogram
axes[1, 1].bar(range(256), eq_hist, color='black', width=1.0)
axes[1, 1].set_title("Equalized Histogram")
axes[1, 1].set_xlabel("Intensity Level")
axes[1, 1].set_ylabel("Pixel Count")
axes[1, 1].set_xlim([0, 255])
axes[1, 1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```



Discussion

In the histogram we can see original image uses only levels 10-80, so spreading it over the full range makes the pollen grains and their spots on grains clearly visible. The equalized histogram is spiky rather than flat because the original bins are only moved apart, not split, and as we can see the same stretch also amplifies the background noise.

Question 6: Foreground-Only Histogram Equalization

Split into HSV, threshold an appropriate plane to build a foreground mask, histogram-equalize only the foreground using cumulative sums, then recombine with the untouched background.

```

In [16]: f = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/q6girl.jpeg')
f_rgb = cv2.cvtColor(f, cv2.COLOR_BGR2RGB)
f_hsv = cv2.cvtColor(f, cv2.COLOR_BGR2HSV)

# split into Hue, Saturation, Value planes

```

```

h, s, v = cv2.split(f_hsv)

# extract foreground mask by thresholding Saturation plane
_, mask = cv2.threshold(s, 30, 255, cv2.THRESH_BINARY)

fg_v = cv2.bitwise_and(v, v, mask=mask)
hist, _ = np.histogram(fg_v[mask > 0], bins=256, range=[0, 256])
cdf = np.cumsum(hist)

# the number of intensity levels
L = 256
MN = np.count_nonzero(mask)

t = np.array([((L - 1) / MN) * cdf[k] for k in range(256)], dtype=np.uint8)

equalized_fg = t[fg_v]
equalized_fg = cv2.bitwise_and(equalized_fg, equalized_fg, mask=mask)

# extract background and add with equalized foreground
mask_inv = cv2.bitwise_not(mask)
bg_v = cv2.bitwise_and(v, v, mask=mask_inv)
v_final = cv2.add(equalized_fg, bg_v)

# recombine the HSV channels and convert back to RGB
hsv_final = cv2.merge([h, s, v_final])
rgb_final = cv2.cvtColor(hsv_final, cv2.COLOR_HSV2RGB)

# Display All Results
fig, ax = plt.subplots(2, 4, figsize=(18, 9))

ax[0, 0].imshow(h, cmap='gray')
ax[0, 0].set_title('Hue Plane')
ax[0, 0].axis('off')

ax[0, 1].imshow(s, cmap='gray')
ax[0, 1].set_title('Saturation Plane')
ax[0, 1].axis('off')

ax[0, 2].imshow(v, cmap='gray')
ax[0, 2].set_title('Value Plane')
ax[0, 2].axis('off')

ax[0, 3].imshow(mask, cmap='gray')
ax[0, 3].set_title('Foreground Mask')
ax[0, 3].axis('off')

ax[1, 0].imshow(fg_v, cmap='gray')
ax[1, 0].set_title('Foreground Only (V)')
ax[1, 0].axis('off')

ax[1, 1].imshow(f_rgb)
ax[1, 1].set_title('Original Image')
ax[1, 1].axis('off')

ax[1, 2].imshow(rgb_final)
ax[1, 2].set_title('Foreground Equalized Result')
ax[1, 2].axis('off')

ax[1, 3].plot(cdf, color='black')
ax[1, 3].set_title('Foreground Cumulative Sum (CDF)')
ax[1, 3].set_xlabel('Intensity Level')
ax[1, 3].set_ylabel('Pixel Count')
ax[1, 3].grid(True)

plt.tight_layout()
plt.show()

# print(f"Raw Foreground CDF Array (Pixel Counts): {cdf}")

# # Print the normalized CDF (mapped from 0.0 to 1.0)
# cdf_normalized = cdf / MN
# print(f"\nNormalized CDF Array: {cdf_normalized}")

# # Print the final intensity mapping table (0 to 255)
# print(f"\nTransformation Mapping Array (LUT): {t}")

```



Discussion

Thresholding saturation separates the chromatic subject from the neutral grey backdrop therefore the skin and hair gain contrast while the background stays exactly as it was. The eyes and the dark jacket have almost no saturation because of that they fall outside the mask and are left unchanged.

Question 7: Sobel Filtering

(a) Use `cv.filter2D` with Sobel kernels. (b) Implement Sobel filtering manually. (c) Exploit the separable kernel property to filter with 1D convolutions.

```
In [17]: image = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/q7AlbertEinstien.jpeg',
cv2.IMREAD_GRAYSCALE) # Replace with your file path

# Sobel kernels for X and Y directions
sobel_y = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]])
sobel_x = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]])

# CV2 Sobel Filtering
im_x = cv2.filter2D(image, cv2.CV_64F, sobel_x, borderType=cv2.BORDER_CONSTANT)
im_y = cv2.filter2D(image, cv2.CV_64F, sobel_y, borderType=cv2.BORDER_CONSTANT)

# -custom 2D Convolution Function
def convolve2d(image, kernel):
    image_height, image_width = image.shape
    kernel_height, kernel_width = kernel.shape
    output = np.zeros((image_height, image_width), dtype=np.float32)

    pad_h = kernel_height // 2
    pad_w = kernel_width // 2
    padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant')

    for i in range(image_height):
        for j in range(image_width):
            value = 0
            for m in range(kernel_height):
                for n in range(kernel_width):
                    value += (padded[i + m, j + n] * kernel[m, n])
            output[i, j] = value

    return output

im_x1 = convolve2d(image, sobel_x)
im_y1 = convolve2d(image, sobel_y)

# Display 2D Sobel Comparisons
fig, axs = plt.subplots(1, 4, figsize=(10, 3))

axs[0].imshow(im_x, cmap='gray')
axs[0].set_title('Sobel X')
axs[0].axis(False)

axs[1].imshow(im_y, cmap='gray')
axs[1].set_title('Sobel Y')
axs[1].axis(False)

axs[2].imshow(im_x1, cmap='gray')
axs[2].set_title('Sobel X (Custom)')
axs[2].axis(False)

axs[3].imshow(im_y1, cmap='gray')
```

```

axs[3].set_title('Sobel Y (Custom)')
axs[3].axis(False)

plt.tight_layout()
plt.show()

# differences between the two methods
print("The differences between Sobel X images:", np.sum(np.abs(im_x - im_x1)))
print("The differences between Sobel Y images:", np.sum(np.abs(im_y - im_y1)))

# y separation
a = np.array([[1], [2], [1]])
b = np.array([1, 0, -1])
im_ya = cv2.filter2D(image, cv2.CV_64F, a)
im_yb = cv2.filter2D(im_ya, cv2.CV_64F, b)

# x separation
a1 = np.array([1, 2, 1])
b1 = np.array([[1], [0], [-1]])
im_xa = cv2.filter2D(image, cv2.CV_64F, a1)
im_xb = cv2.filter2D(im_xa, cv2.CV_64F, b1)

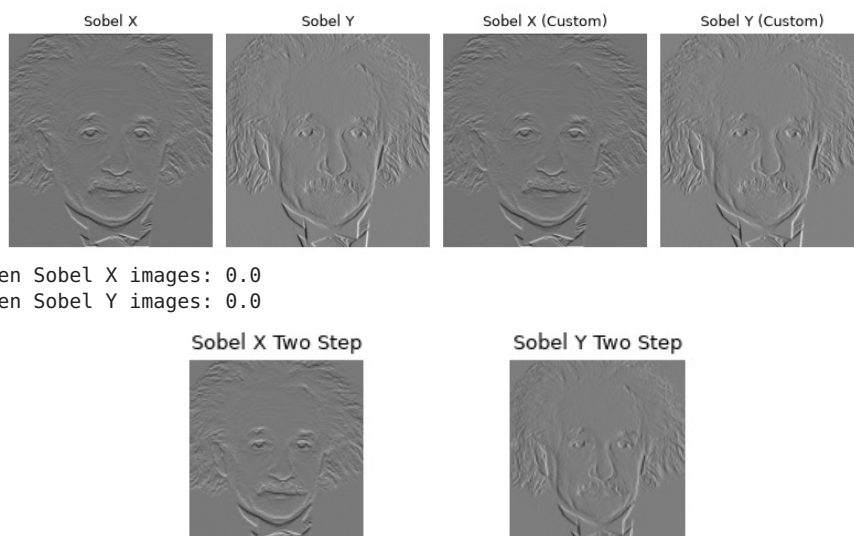
# display 1D Separable Sobel Comparisons
fig, axs = plt.subplots(1, 2, figsize=(6, 2))

axs[0].imshow(im_xb, cmap='gray')
axs[0].set_title('Sobel X Two Step')
axs[0].axis(False)

axs[1].imshow(im_yb, cmap='gray')
axs[1].set_title('Sobel Y Two Step')
axs[1].axis(False)

plt.tight_layout()
plt.show()

```



The differences between Sobel X images: 0.0
The differences between Sobel Y images: 0.0

Discussion

In this both filters responded only to intensity change, so flat regions come out mid-grey and only edges appear — one image picks up horizontal features (moustache, collar) while the other vertical ones (hair strands, face outline). The manual result is identical to `filter2D` (difference 0.0), and the separable two-pass version reproduces it as well.

Question 8: Image Zooming (Nearest-Neighbor and Bilinear)

Write a zoom function supporting nearest-neighbor and bilinear interpolation for scale factor s in $(0, 10]$. Evaluate using normalized SSD against the original when scaling up by 4x.

```

In [18]: def zoom_image(img, s, method='bilinear'):
    if not (0 < s <= 10):
        raise ValueError(f"s must be in (0, 10], got {s}")

    h_orig, w_orig = img.shape[:2]
    h_new = max(1, int(round(h_orig * s)))
    w_new = max(1, int(round(w_orig * s)))

```

```

is_color = img.ndim == 3

x_new, y_new = np.meshgrid(np.arange(w_new), np.arange(h_new))

# Map new pixel coordinates back to original image coordinates
x_orig = (x_new + 0.5) / s - 0.5
y_orig = (y_new + 0.5) / s - 0.5

img_float = img.astype(np.float32)

if method == 'nearest':
    x_n = np.clip(np.round(x_orig).astype(int), 0, w_orig - 1)
    y_n = np.clip(np.round(y_orig).astype(int), 0, h_orig - 1)
    out = img_float[y_n, x_n]

elif method == 'bilinear':
    x0 = np.clip(np.floor(x_orig).astype(int), 0, w_orig - 1)
    y0 = np.clip(np.floor(y_orig).astype(int), 0, h_orig - 1)
    x1 = np.clip(x0 + 1, 0, w_orig - 1)
    y1 = np.clip(y0 + 1, 0, h_orig - 1)

    dxf = np.clip(x_orig - x0, 0, 1)
    dyf = np.clip(y_orig - y0, 0, 1)
    dx = dxf[..., np.newaxis] if is_color else dxf
    dy = dyf[..., np.newaxis] if is_color else dyf

    Ia, Ib = img_float[y0, x0], img_float[y0, x1]
    Ic, Id = img_float[y1, x0], img_float[y1, x1]
    out = (1 - dx) * (1 - dy) * Ia + dx * (1 - dy) * Ib + (1 - dx) * dy * Ic + dx * dy * Id

else:
    raise ValueError("method must be 'nearest' or 'bilinear'")

return np.clip(out, 0, 255).astype(np.uint8)

def compute_normalized_ssd(img1, img2):
    # function to compute the normalized sum of squared differences (SSD) between two images
    h = min(img1.shape[0], img2.shape[0])
    w = min(img1.shape[1], img2.shape[1])
    a, b = img1[:h, :w], img2[:h, :w]
    diff = a.astype(np.float64) - b.astype(np.float64)
    return np.sum(diff ** 2) / diff.size

# all images
image_pairs = [
    {"name": "image1", "small": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im01small.png",
    "orig": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im01.png"},
    {"name": "image2", "small": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im02small.png",
    "orig": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im02.png"},
    {"name": "image3", "small": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im03small.png",
    "orig": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im03.png"},
    {"name": "image4", "small": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im04small.jpg",
    "orig": "/home/lasan-perera/University(Aca)/CV Assignment 1/images/im04.jpg"},
]
SCALE_FACTOR = 4.0
# Grayscale or Color comparison (set COLOR = False for grayscale)
COLOR = True

flag = cv2.IMREAD_COLOR if COLOR else cv2.IMREAD_GRAYSCALE
results = []

for pair in image_pairs:
    orig_img = cv2.imread(pair["orig"], flag)
    small_img = cv2.imread(pair["small"], flag)
    if orig_img is None or small_img is None:
        print(f"[skip] could not read files for '{pair['name']}' -- check paths")
        continue

    zn = zoom_image(small_img, SCALE_FACTOR, method='nearest')
    zb = zoom_image(small_img, SCALE_FACTOR, method='bilinear')

    ssd_n = compute_normalized_ssd(orig_img, zn)
    ssd_b = compute_normalized_ssd(orig_img, zb)
    results.append({"name": pair["name"], "orig": orig_img, "zn": zn, "zb": zb,
                    "ssd_n": ssd_n, "ssd_b": ssd_b})
    print(f"{pair['name']:12s} NN SSD = {ssd_n:9.3f} Bilinear SSD = {ssd_b:9.3f}")

if results:
    avg_n = np.mean([r["ssd_n"] for r in results])

```

```

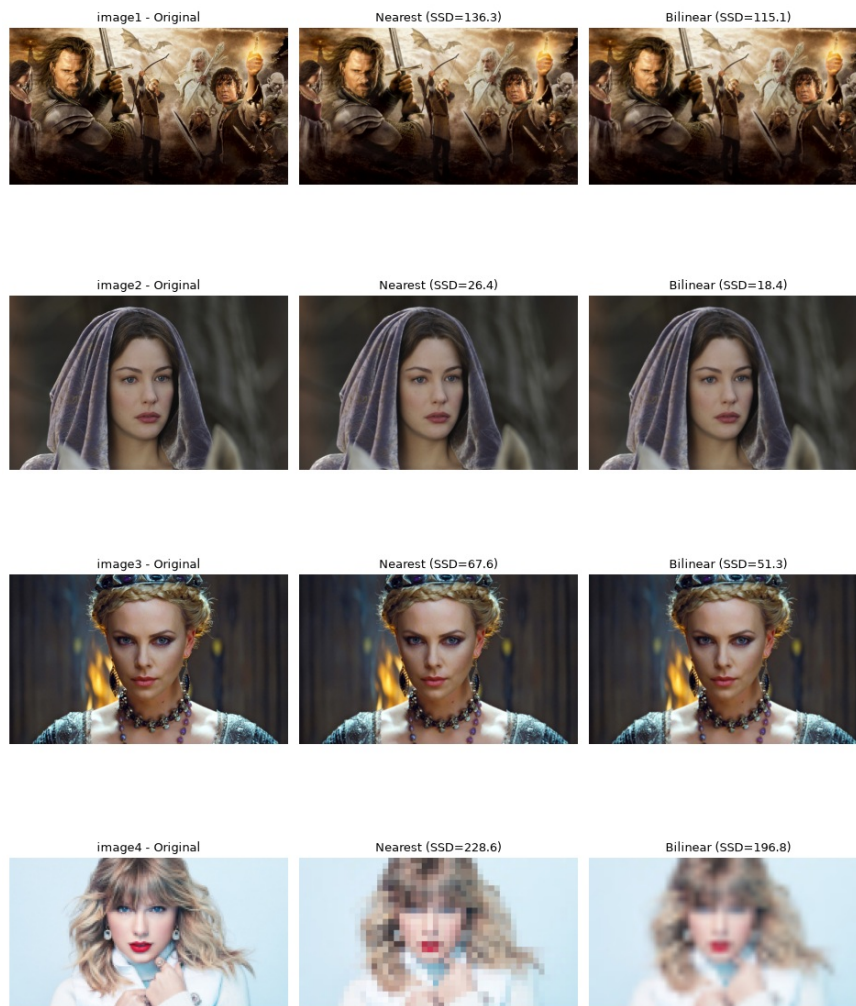
avg_b = np.mean([r["ssd_b"] for r in results])
print(f"{'AVERAGE':12s}  NN SSD = {avg_n:9.3f}   Bilinear SSD = {avg_b:9.3f}")

# print the results in a tabular format
n = len(results)
fig, axes = plt.subplots(n, 3, figsize=(12, 4 * n))
if n == 1:
    axes = axes[np.newaxis, :]
cmap = None if COLOR else 'gray'
for row, r in enumerate(results):
    for col, key, title in [(0, "orig", "Original"),
                           (1, "zn", f"Nearest (SSD={r['ssd_n']:.1f})"),
                           (2, "zb", f"Bilinear (SSD={r['ssd_b']:.1f})")]:
        img = r[key] if not COLOR else cv2.cvtColor(r[key], cv2.COLOR_BGR2RGB)
        axes[row, col].imshow(img, cmap=cmap)
        axes[row, col].set_title(f"{r['name']} - {title}" if col == 0 else title)
        axes[row, col].axis("off")

plt.tight_layout()
plt.savefig("zoom_comparison_grid.png", dpi=130)
plt.show()

```

image1	NN SSD =	136.269	Bilinear SSD =	115.126
image2	NN SSD =	26.446	Bilinear SSD =	18.397
image3	NN SSD =	67.582	Bilinear SSD =	51.320
image4	NN SSD =	228.622	Bilinear SSD =	196.766
AVERAGE	NN SSD =	114.730	Bilinear SSD =	95.402



Discussion

Nearest neighbour copies each pixel into a 4×4 block, giving blocky edges and jagged diagonals while bilinear averages four neighbours and looks smooth therefore its SSD is lower on every image (95.40 vs 114.73 on average). Then we can see both errors stay large because the detail destroyed when the image was originally downsampled cannot be recovered by any interpolation.

Question 9: GrabCut Segmentation and Background Blur

(a) Segment the flower image with cv.grabCut, showing mask, foreground, and background. (b) Blur the background to produce a bokeh-style enhanced image. (c) Discuss the dark rim artifact.

```
In [19]: img = cv2.imread('/home/lasan-perera/University(Aca)/CV Assignment 1/images/q9sunFlower.jpeg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Grabcut variables
mask = np.zeros(img.shape[:2], np.uint8)
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)

h, w = img.shape[:2]
rect = (int(0.1 * w), int(0.1 * h), int(0.9 * w), int(0.9 * h)) #A rough boundary box around the
sunflower

# Grabcut segmentation
cv2.grabCut(img, mask, rect, bgdModel, fgdModel, 5, cv2.GC_INIT_WITH_RECT)

# binary mask
mask2 = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')
foreground = img_rgb * mask2[:, :, np.newaxis] # Foreground image
background = img_rgb * (1 - mask2[:, :, np.newaxis]) # Background image

blurred = cv2.GaussianBlur(background, (35, 35), 0)
mask3 = mask2[:, :, np.newaxis]
result = img_rgb * mask3 + blurred * (1 - mask3)

# Results Visualization
fig, axes = plt.subplots(1, 5, figsize=(12, 3))

axes[0].imshow(mask2, cmap='gray')
axes[0].set_title("Segmentation Mask")
axes[0].axis("off")

axes[1].imshow(foreground)
axes[1].set_title("Foreground")
axes[1].axis("off")

axes[2].imshow(background)
axes[2].set_title("Background")
axes[2].axis("off")

axes[3].imshow(img_rgb)
axes[3].set_title("Original")
axes[3].axis("off")

axes[4].imshow(result)
axes[4].set_title("Background Blurred, Foreground Sharp")
axes[4].axis("off")

plt.tight_layout()
plt.show()
```



Discussion

The flower segments cleanly because the yellow petals and green foliage occupy well separated colour ranges. The dark rim appears because the foreground is blacked out before the background is blurred because of that near the boundary the Gaussian averages in those black pixels and darkens a band about one kernel radius wide.

Question 10: Bilateral Filtering

(a) Use `cv.bilateralFilter` and compare against a similarly-sized Gaussian blur. (b) Implement your own bilateral filter, compare against OpenCV's version with a quantitative measure.

```
In [20]: img = cv2.imread("/home/lasan-perera/University(Aca)/CV Assignment 1/images/q10Mountains.jpeg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# parameters
sigma_s = 75
sigma_r = 75
```

```

ksize = (9, 9)
d = 9

#Builtin Bilateral and Gaussian Filters
bilateral = cv2.bilateralFilter(img_rgb, d=d, sigmaColor=sigma_r, sigmaSpace=sigma_s)
gaussian = cv2.GaussianBlur(img_rgb, ksize, sigmaX=sigma_s / 25)

# Comparison Visualization
fig, axes = plt.subplots(1, 3, figsize=(15, 3))
axes[0].imshow(img_rgb)
axes[0].set_title("Original")
axes[0].axis("off")

axes[1].imshow(gaussian)
axes[1].set_title(f"Gaussian Blur (kernel={ksize})")
axes[1].axis("off")

axes[2].imshow(bilateral)
axes[2].set_title(rf"Bilateral Filter ($\sigma_s$={sigma_s}, $\sigma_r$={sigma_r})")
axes[2].axis("off")

plt.tight_layout()
plt.show()

# Custom Bilateral Filter
def my_bilateral_filter(img, d, sigma_s, sigma_r, eps=1e-8):
    img = np.asarray(img, dtype=np.float64)
    h, w, c = img.shape
    radius = d // 2
    padded = cv2.copyMakeBorder(img, radius, radius, radius, radius, cv2.BORDER_REFLECT)

    ax = np.arange(-radius, radius + 1)
    xx, yy = np.meshgrid(ax, ax)
    spatial_weight = np.exp(-(xx**2 + yy**2) / (2 * sigma_s**2))

    result_acc = np.zeros((h, w, c))
    weight_sum = np.zeros((h, w, 1))

    for i in range(d):
        for j in range(d):
            shifted = padded[i:i+h, j:j+w, :]
            diff = shifted - img
            range_dist_sq = np.sum(diff**2, axis=2, keepdims=True)
            exponent = np.clip(-range_dist_sq / (2 * sigma_r**2 + eps), -700, 0)
            range_weight = np.exp(exponent)

            weight = spatial_weight[i, j] * range_weight
            result_acc += weight * shifted
            weight_sum += weight

    output = result_acc / (weight_sum + eps)
    return output

# Custom Bilateral Filter
my_bilateral = my_bilateral_filter(img_rgb, d, sigma_s, sigma_r)
my_bilateral_uint8 = np.clip(my_bilateral, 0, 255).astype(np.uint8)

mse_value = np.mean((my_bilateral_uint8.astype(np.float64) - bilateral.astype(np.float64))**2)
print("MSE between two:", mse_value)

# image comparison visualization
fig, axes = plt.subplots(1, 4, figsize=(20, 4))
axes[0].imshow(img_rgb)
axes[0].set_title("Original")
axes[0].axis("off")

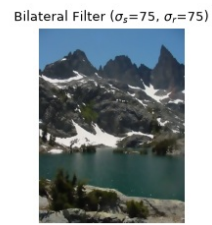
axes[1].imshow(gaussian)
axes[1].set_title(f"Gaussian Blur ({d}x{d})")
axes[1].axis("off")

axes[2].imshow(bilateral)
axes[2].set_title("OpenCV Bilateral")
axes[2].axis("off")

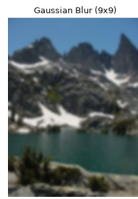
axes[3].imshow(my_bilateral_uint8)
axes[3].set_title("Custom Bilateral")
axes[3].axis("off")

plt.tight_layout()
plt.show()

```



MSE between two: 28.909178208679595



Discussion

The Gaussian softens the ridge line and shoreline along with the texture, whereas the bilateral filter smooths only inside regions and keeps those edges sharp, because neighbours on the other side of an edge receive almost zero weight. The custom implementation matches OpenCV's visually (MSE 28.90917 about 5 intensity levels), the small gap coming from different range-distance and neighbourhood conventions.